EVALUACIÓN PARCIAL 2

Gestión de Desarrollos de Software

Carrera: Tecnicatura Universitaria en Programación a Distancia (UTN)
Alumno: Nicolás Viruel — Matrícula 100634 — Comisión TUPaD
Fecha: 16/10/2026
Modalidad: Individual

Parte 1: Planificación, Entregables y Camino Crítico (CPM)

1. Cálculo del Camino Crítico (CPM)

a. Tiempos tempranos (ES, EF) y tardíos (LS, LF)

Convención: ES = inicio temprano, EF = fin temprano, LS = inicio tardío, LF = fin tardío, H = holgura = LS − ES = LF − EF. Tiempos en días, contados desde el día 0.

Pasada hacia adelante (ES = máximo EF de las predecesoras; EF = ES + duración) y pasada hacia atrás (LF = mínimo LS de las sucesoras; LS = LF − duración; para F, LF = duración total del proyecto).

Código	Actividad	Prec.	Dur.	ES	EF	LS	LF	H
A	Levantamiento de requisitos y aprobación	—	2	0	2	0	2	0
B	Diseño de la Arquitectura de Software	A	3	2	5	2	5	0
C	Desarrollo de la API REST (Backend)	B	5	5	10	5	10	0
D	Diseño de Interfaz y UI/UX (Frontend)	B	3	5	8	7	10	2
E	Pruebas de Integración y Seguridad	C, D	4	10	14	10	14	0
F	Despliegue en producción	E	2	14	16	14	16	0

Detalle del cálculo:

- A: ES = 0; EF = 0 + 2 = 2.
- B: ES = EF(A) = 2; EF = 2 + 3 = 5.
- C: ES = EF(B) = 5; EF = 5 + 5 = 10.
- D: ES = EF(B) = 5; EF = 5 + 3 = 8.
- E: ES = máx(EF(C), EF(D)) = máx(10, 8) = 10; EF = 10 + 4 = 14.
- F: ES = EF(E) = 14; EF = 14 + 2 = 16.
- Hacia atrás: F: LF = 16, LS = 14. E: LF = LS(F) = 14, LS = 10. C: LF = LS(E) = 10, LS = 5. D: LF = LS(E) = 10, LS = 10 − 3 = 7. B: LF = mín(LS(C), LS(D)) = mín(5, 7) = 5, LS = 2. A: LF = LS(B) = 2, LS = 0.

b. Duración total esperada del proyecto

16 días (EF de la última actividad, F).

c. Ruta Crítica y holgura de D

- Ruta Crítica: A → B → C → E → F (2 + 3 + 5 + 4 + 2 = 16 días). Son las actividades con holgura cero: cualquier retraso en ellas se traslada 1:1 a la fecha de fin del proyecto.
- Holgura de D: H(D) = LS − ES = 7 − 5 = 2 días (equivalente: LF − EF = 10 − 8 = 2). D corre en paralelo a C y es 2 días más corta, por lo que puede iniciar hasta el día 7 (o demorarse hasta 2 días) sin atrasar el inicio de E ni la fecha final. D es la única actividad no crítica del proyecto.

2. Hitos y Entregables

Criterio (material de la unidad 4): un hito es un evento significativo que marca el cumplimiento de una etapa; no tiene duración ni consume recursos y es verificable (ej.: "documento de requisitos aprobado"). Un entregable es un producto o resultado tangible; es interno cuando está dirigido a los equipos o stakeholders de la propia organización, y externo cuando se presenta al cliente y materializa el valor del proyecto hacia afuera.

Elemento	**Clasificación**	**Justificación**
1. Documento de Arquitectura Técnica en formato Markdown	Entregable interno	Es un producto tangible (artefacto de documentación técnica que vive en docs/ del repositorio) cuyo destinatario es el equipo de desarrollo, QA y mantenimiento de LogiExpress, no el cliente final. Resulta de la actividad B.
2. "Aprobación formal del documento de requisitos por parte del cliente"	Hito	Es un evento puntual y verificable (acta/aprobación), sin duración ni consumo de recursos, que marca el cierre de la actividad A y habilita el inicio del diseño (B). Coincide con el ejemplo clásico del material: "documento de requisitos aprobado".
3. Módulo web de rastreo funcionando y desplegado para el cliente final	Entregable externo	Es el producto final del proyecto, destinado al cliente (operador logístico); es lo que "gana o conserva" al cliente y materializa el objetivo del contrato. Se concreta con la actividad F (despliegue en producción).

Parte 2: Gestión Dinámica e Inteligencia Artificial Predictiva

Situación: durante el Sprint 2 la actividad C (Desarrollo Backend), que está en la ruta crítica, se estanca por una complicación técnica no prevista en la integración con la base de datos.

1. Métrica que analiza la IA Predictiva para detectar el bloqueo

La métrica operativa es el Lead Time (tiempo medio de resolución de las tareas), analizado junto con el tiempo que la tarea lleva sin cambios de estado en el tablero. La IA predictiva compara el comportamiento actual de la tarea con el historial de ejecución del equipo: si "Desarrollo de la API REST" suele resolverse en 5 días y ya lleva más tiempo sin avanzar de estado (por ejemplo, estancada en "In Progress"), el sistema la identifica como un cuello de botella en la ruta crítica y dispara una alerta de riesgo de forma autónoma, antes de que el desvío llegue a comprometer la fecha final (identificación temprana de cuellos de botella).

Complementariamente, cruza la Velocity histórica de los sprints anteriores con el trabajo pendiente para recalcular la probabilidad real de cumplir el hito en la fecha pactada (forecast estadístico). En el caso de LogiExpress, al estar C en la ruta crítica (holgura 0), cualquier día de estancamiento se refleja directamente en la proyección de la fecha de despliegue.

2. Crash Costing asistido por IA

En qué consiste: el Crash Costing (compresión del cronograma) busca acortar la duración total del proyecto agregando recursos a las actividades de la ruta crítica al menor costo adicional posible. Como solo tiene sentido comprimir tareas críticas (reducir D, que tiene holgura, no acortaría el proyecto), el PM elegiría reforzar C. La IA evalúa, mediante simulaciones "what-if", distintas combinaciones de refuerzo (por ejemplo, sumar un desarrollador senior a C o dividir la tarea en dos) y muestra en segundos el impacto de cada alternativa sobre la fecha final y el presupuesto, proponiendo la opción con mejor relación costo/tiempo ganado.

Dos criterios técnicos/operativos que consideraría la IA para reasignar recursos:

1. Seniority y especialidad técnica: no mueve "personas" genéricas, sino que busca quién tiene la habilidad específica requerida por la tarea bloqueada. En este caso, un desarrollador con experiencia en integración de bases de datos y APIs REST, para que el refuerzo realmente destrabe el problema y no agregue costo de coordinación sin beneficio.
2. Gestión de restricciones y disponibilidad: considera simultáneamente la disponibilidad real de la persona (carga actual, holgura de sus tareas), ubicación y zona horaria, y la disponibilidad de materiales o licencias (por ejemplo, acceso al entorno de base de datos). Un candidato natural es un integrante de D, que tiene 2 días de holgura, siempre que la reasignación no convierta a D en crítica.

Parte 3: Artefactos de Software y Docs-as-Code

1. Estructura de Artefactos

Estructura estándar de proyecto (material de la unidad 5): src/ código fuente, docs/ documentación técnica, tests/ pruebas, config/ configuración, assets/ recursos y README.md información general.

Elemento	**Ubicación**	**Comentario**
a. Código fuente del Backend y Frontend	src/ (por ejemplo src/backend/ y src/frontend/)	Carpeta del código fuente del sistema; se separa por componente para mantener la estructura limpia.
b. Diagrama de arquitectura y documentación técnica	docs/ (por ejemplo docs/arquitectura.md y el diagrama como código o imagen en docs/)	Documentación técnica versionada junto al código, en texto plano (Markdown), según la filosofía Docs-as-Code.
c. Casos de prueba automatizados	tests/	Pruebas del sistema (unitarias, de integración y de seguridad, como las de la actividad E), ejecutables por el pipeline de CI/CD.
d. Descripción general del proyecto y guía rápida de inicio	README.md (en la raíz del repositorio)	Información general del proyecto, propósito y pasos de inicio rápido; es el punto de entrada para cualquier persona que llegue al repositorio.

2. Pilar de Docs-as-Code que publica la documentación automáticamente

El pilar es la Automatización CI/CD (Integración Continua y Despliegue Continuo). Es el mecanismo que, cada vez que se aprueba e integra un cambio (merge del Pull Request), ejecuta un pipeline que valida la documentación (enlaces, formato, ortografía) y la construye y publica automáticamente en el servidor o sitio de documentación, sin intervención manual. Los otros pilares cumplen roles distintos: Git versiona, Markdown da el formato de texto plano y los Pull Requests aportan la revisión colaborativa previa a la aprobación.

Parte 4: Manuales Tecnológicos, POEs e IA Generativa

1. Tipos de Manuales

Destinatario	Tipo de documento	Justificación
A. Administrador de Servidores que configura variables de entorno, puertos de red y dependencias para poner en funcionamiento la API por primera vez	Manual de instalación	Es el documento que guía al administrador o técnico en la puesta en funcionamiento del sistema por primera vez. Su estructura coincide exactamente con lo que necesita: requisitos previos y dependencias, pasos de instalación, configuración inicial (variables de entorno, base de datos, puertos de red), verificación y solución de problemas comunes. Debe ser extremadamente preciso y no dar nada por supuesto.
B. Cliente final (operador logístico) que necesita aprender a usar las pantallas para buscar un paquete	Manual de usuario	Explica paso a paso cómo usar las funcionalidades del producto, en lenguaje simple y sin jerga técnica, asumiendo que el lector no programa. Incluye descripción general, instrucciones para las tareas frecuentes (por ejemplo, "buscar un envío por número de tracking"), capturas de pantalla anotadas, FAQ y glosario.

2. Uso de IA en Redacción Técnica

Dos riesgos o limitaciones críticas de publicar el manual de instalación generado por IA sin revisión humana previa:

1. Puede inventar pasos o datos incorrectos (alucinaciones): si la IA no tiene suficiente contexto del sistema real, genera instrucciones que parecen lógicas pero son erróneas: nombres de variables de entorno, puertos, versiones de dependencias o comandos que no corresponden a LogiExpress. En un manual de instalación, un paso inventado hace que la API directamente no levante o quede mal configurada.
2. Puede omitir advertencias críticas y desconoce el contexto de la organización: la IA no siempre sabe qué pasos pueden causar pérdida de datos o problemas de seguridad (por ejemplo, exponer un puerto, publicar credenciales de ejemplo o ejecutar una migración destructiva sobre la base de producción), ni conoce términos internos, flujos de aprobación o configuraciones particulares de la infraestructura de la empresa. Publicar sin revisión traslada ese riesgo al administrador que lo siga al pie de la letra.

Mitigación recomendada (buenas prácticas del material): revisar y validar técnicamente el borrador, agregar el contexto interno manualmente y testear las instrucciones haciendo que alguien que nunca instaló el sistema las siga paso a paso antes de publicar; luego mantener el manual en el repositorio bajo Docs-as-Code.